

Leaper: target memory and the LSTM runs

What worked, what changed when the target was hidden, why the LSTM runs struggled, and the questions we need Fable to answer.

What we are trying to do

The intended game behavior has three distinct competencies:

1. Locomotion and local collision avoidance

Move decisively, steer, avoid obstacles, escape contact, and avoid freezing.

2. Search and discovery

When the target is not visible, explore enough of the arena to bring it into the visual cone.

3. Tracking and pursuit

After seeing the target, continue toward it through temporary occlusion. Later, when the target moves, estimate where it is going and intercept or reacquire it.

Earlier experiments mostly solved competency 1 while giving competency 3 privileged information. PPO_27 onward began testing competency 2 honestly. A moving target will add temporal prediction to competency 3.

This separation matters because "the robot reached the target" can mean very different things:

- **Known-target navigation:** the policy always receives the true target direction and distance.
- **Hidden static target:** the policy receives no target direction/distance until line of sight, then receives last-seen information for a finite period.
- **Moving visible target:** current bearing/range changes over time and contains a velocity signal.
- **Moving occluded target:** the controller must maintain a belief about position and velocity while observations are absent.

The 93% PPO_25 score belongs to the first task. The 64-71% PPO_27-29 scores belong to the second, harder task. A future moving-target score will represent a third or fourth task and should have its own fixed evaluation suite.

How we got here

This is a deliberately compressed history. Its purpose is to show what capabilities were already established before the hidden-target and LSTM work.

Run family	Main change	Deterministic result	Lesson carried forward
PPO_1-2	Initial body-only PPO and longer training	PPO_2 about 57-60%	Basic continuous-control navigation can learn.
PPO_3-7	Added leg collisions and several reward experiments	mostly poor/incomplete	The 19-point collision model made the task substantially harder.
PPO_8-10	Net-distance reward, diagnostic state, signed throttle	0%, 0%, then 5%	Zero-throttle collapse and collision loops were separate problems.
PPO_11	Added eight obstacle rays	39-69% across seeds	Seeing obstacles was the decisive improvement.
PPO_12	Decoupled rotation and translation collision resolution	65%	A contacting robot must retain a way to rotate or move out.
PPO_13	Larger randomized arena, 10 obstacles, 300k	85%	Navigation generalized, although the arena was relatively open.

Run family	Main change	Deterministic result	Lesson carried forward
PPO_14-16	Dense 51-obstacle arena; collision penalty test; terminal stuck rule	54%, 51%, 64%	A flat collision penalty caused timidity; the terminal stuck rule removed deep freezes.
PPO_17	200-degree body-aware sectors	8%	Restricted vision plus reverse movement produced a poor sensor/action match.
PPO_18	270-degree thin rays, range 28, forward-only	75%	"Move inside what you can see" restored decisive behavior.
PPO_19-21	One-layer recurrent LSTM at three learning rates on the known-target task	71%, incomplete, 69%	Full recurrence did not beat the 75% MLP baseline and was less stable.
PPO_22	Widened feed-forward network to 256x256	74%	More raw capacity was not the missing ingredient.
PPO_23	Increased rays from 8 to 16	73-87%, mean 81.5%	Better angular resolution helped but training seeds varied in decisiveness.
PPO_24/26	Increased ray range from 28 to 45	80%; then 80-84%	Longer range compressed near-field detail and increased hesitation. Range 28 remained preferable.
PPO_25	Added idle penalty to the 16-ray, range-28 design	89-93%, mean 91.5%	The fully informed navigation target was met reliably.

The project has therefore already tested several tempting explanations - more network width, longer sight, larger collision penalty, and generic recurrent memory - without finding a universal improvement. The strongest repeated lesson is that the observation/action contract and reward loopholes matter more than merely enlarging the network.

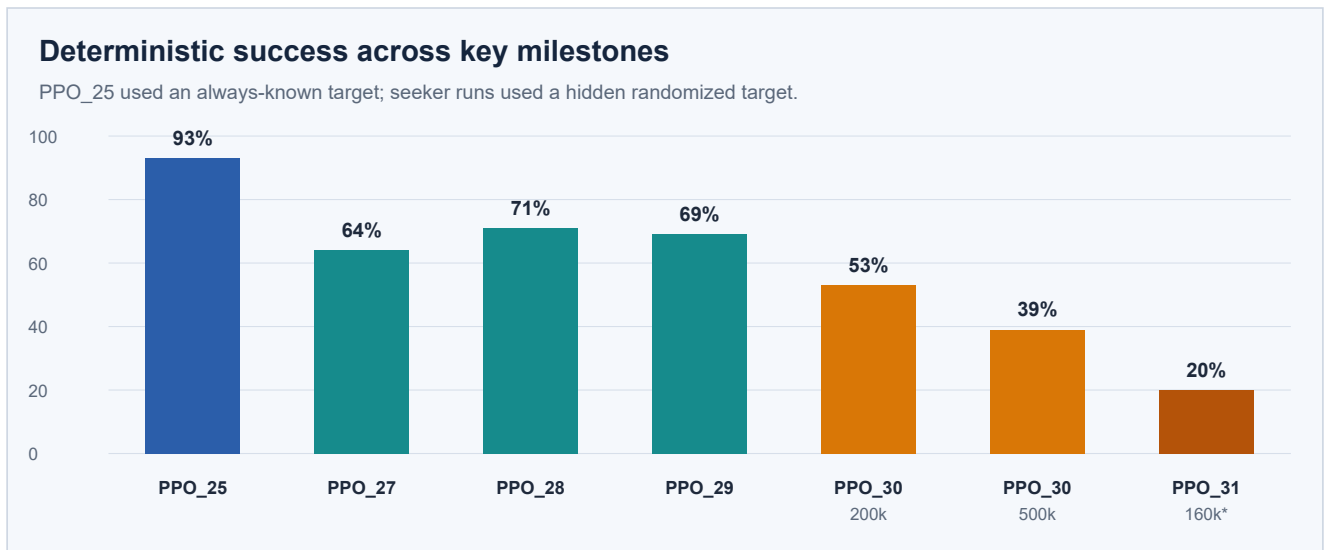


Figure 1. PPO_25 is a known-target navigation exam. PPO_27 onward are hidden-target seeker exams and are not directly comparable.

*PPO_31 is a 25-episode checkpoint, not a final 100-episode exam.

Best run before target search: PPO_25

What PPO_25 solved

PPO_25 learned to move toward a known goal through dense randomized obstacle fields. It did **not** visually identify or search for the goal. The policy was given the exact target direction and normalized distance on every step.

The target was fixed at $(-40, -10)$ in a square arena with $\text{WORLD_LIMIT} = 93.75$, giving a total width of 187.5 units. Each episode used 51 randomized circular obstacles with a reachability guard. The robot spawned at randomized valid poses, so the policy still had to solve many unseen navigation layouts.

Observation: 26 values

Indices	Meaning	Notes
0-1	normalized robot world position $x / 93.75, z / 93.75$	Absolute self-localization.
2-3	exact unit direction to target in world x/z	Always available; this is the privileged target information.
4	target distance divided by the maximum arena diagonal	Always available and clipped to $[0, 1]$.
5-6	$\sin(\text{yaw}), \cos(\text{yaw})$	Continuous heading without angle wrap discontinuity.
7	previous-step collision flag	1 after collision, otherwise 0.
8	previous throttle	Short one-step action context.
9	previous turn	Short one-step action context.
10-25	16 obstacle/wall rangefinder rays	Thin point rays from the body centre.

The 16 rays cover a **270-degree forward cone**. They sit at the centres of 16 equal sectors, so adjacent ray centres are 16.875 degrees apart. The rear 90-degree wedge is blind. Each ray reports the distance to the nearest obstacle surface or arena wall divided by **28 units**, clipped to $[0, 1]$. A reading of 1 means clear for at least 28 units; 0 means the origin is effectively at the surface.

These are geometric sensors, not pixels or a learned visual recognizer. They also do not represent the full swept volume of all six legs; they are thin centre-origin rays.

Action and physics

Action	Policy range	Physical meaning
throttle	$[0, 1]$	Forward-only; distance per step is throttle times 0.75 units.
turn	$[-1, 1]$	Yaw change is turn times 18 degrees per step.

The full intended turn-and-move pose is collision-checked. If blocked, rotation and translation are resolved independently, allowing rotation in place when possible. Collision checking uses 19 planar samples:

- one body circle, radius 0.75;
- three samples on each of six legs at radial distances 1.2, 2.2, and 3.24;
- each leg sample has radius 0.24.

Reward system

PPO_25's reward was a known-target navigation reward:

Term	Value / rule	Intended effect
distance progress	$0.2 * (\text{previous distance} - \text{current distance})$	Reward actual movement toward the known target and penalize retreat.
time	-0.01 every step	Prefer shorter routes and discourage endless episodes.
idle	-0.04 after more than 3 consecutive steps with throttle below 0.1	Remove the timid "standing still is safe" strategy while allowing a brief pivot.
collision	-0.18 per blocked intended move	Discourage contact.
stuck failure	one-time -10, terminal after 40 consecutive collision/no-progress steps	Make deep wedges a clearly bad terminal outcome.
goal	+25 once	Reward reaching within 2.15 units: target radius 1.4 plus agent radius 0.75.

There were no alive, upright, energy, gait-smoothness, balance, fall, search, visibility, or target-memory rewards.

Brain architecture

Algorithm: Stable-Baselines3 PPO with MlpPolicy.

```
26-number observation
|-- actor:  Linear 26->64 -> Tanh -> Linear 64->64 -> Tanh
|           -> Linear 64->2 action mean + 2 learned log standard deviations
|
|-- critic: Linear 26->64 -> Tanh -> Linear 64->64 -> Tanh
|           -> Linear 64->1 state value
```

The actor and critic MLPs are separate. During training, continuous actions are sampled from a Gaussian described by the actor mean and learned state-independent log standard deviation. Deterministic evaluation and deployment use the action mean. The verified saved policy has **11,973 trainable parameters**.

PPO training configuration

Setting	Value
requested transitions	500,000
collected transitions	507,904
parallel environments	8
steps per environment per rollout	1,024
rollout size	8,192 transitions
batch size	256
epochs per rollout	10
learning rate	3e-4
gamma	0.995
GAE lambda	0.95
entropy coefficient	0.01
PPO clip range	0.2, library default
value coefficient	0.5, library default
max gradient norm	0.5, library default
training seed for champion	911743
evaluation	100 deterministic episodes, seeds 10000-10099

Because PPO finishes full 8,192-transition rollouts, requesting 500k produced 507,904 transitions.

Results

Metric	Deterministic 100-maze exam	Latest 100 stochastic training episodes
success	93%	91%
timeout	7%	9%
mean reward	35.06	33.51
mean net target progress	72.10	71.61
average episode length	135.74	151.06
forward steps	99.50%	93.52%
stopped steps	0.50%	6.48%
mean throttle	0.984	0.869
collision steps	2.17%	4.57%
mean longest collision streak	2.84	4.14
worst collision streak	40	96

Across four seeds, the same recipe scored **89%, 92%, 92%, and 93%**, a mean of **91.5%**. The 93% champion was also tested on 1,000 different held-out mazes and scored **91.3%**, with a reported 95% confidence interval of 89.6-93.0%. This strongly supports genuine obstacle-navigation generalization.

Approximate recorded wall-clock time from artifact timestamps for the champion run was **30.6 minutes** on the machine used. This is an artifact-time estimate, not a benchmark controlled for other system load.

Correct interpretation

PPO_25 is strong evidence that Leaper can learn local navigation and obstacle avoidance. It is not evidence that Leaper can discover a hidden target. Its exact target vector simplified the problem into "reach this known point." Any report that compares PPO_25's 93% directly to a seeker score without this caveat will overstate the LSTM regression.

First hidden-target run: PPO_27

Task change

PPO_27 removed permanently available target coordinates. The target became randomized each episode and was exposed to the policy only when all of the following were true:

- it was inside the 270-degree forward field of view;
- it was within the 28-unit target-sensing range;
- no obstacle occluded the line segment to it.

Target identity remained semantic: the environment knows which game object is the target. This was not pixel-based detection.

The arena was reduced from a 187.5-unit width and 51 obstacles to a **62.5-unit width** (`WORLD_LIMIT=31.25`) and **6 obstacles** to retain roughly comparable obstacle density. PPO_27 used a 500-step episode cap, movement speed 0.75, and turn speed 18 degrees.

Observation redesign while retaining 26 values

Indices	PPO_25 meaning	PPO_27 seeker meaning
0	normalized x position	target currently visible flag
1	normalized z position	target-memory age, divided by 120
2-3	exact target direction	zero before sight; direction to visible or last-seen target after sight
4	exact target distance	1 before sight; normalized visible or remembered distance after sight
5-9	heading, collision, prior actions	retained
10-25	16 obstacle rays	retained

The environment stores the last-seen target position. When the target is occluded, the policy receives direction and distance to that stored point. The age channel rises toward 1 over **120 steps**. Importantly, this is already explicit non-neural memory.

Transfer from PPO_25

PPO_27 loaded the PPO_25 champion to preserve useful navigation weights. Because input channels 0 and 1 changed meaning, their first-layer columns were zeroed. The existing Adam optimizer state was cleared. Channels 2-25 remained structurally compatible enough to retain goal-direction, heading/action/collision, and obstacle-ray weights.

This was partial transfer, not a claim that the old task and new task were identical. The actor still had to learn how to act before the target was discovered and how to use visibility/memory age.

Reward system

Term	Rule
post-sight visible progress	clipped distance change times 0.2
pre-sight search progress	positive-only improvement beyond best-ever distance times 0.1
sight/reacquisition	+0.5 whenever sight was regained in PPO_27
new spatial cell	+0.01
new cell/heading view	+0.002
time	-0.002 per step
idle	-0.04 after 15 repeated no-move/no-new-view steps
collision	-0.18
stuck	terminal -10 after 40 blocked steps
goal	+25

The positive-only pre-sight distance term used private target distance for reward shaping but did not reveal coordinates in the observation. This still gives the learner a weak directional training signal even though the deployed policy cannot read the target position.

Training and results

The architecture remained the 11,973-parameter 64x64 MLP. PPO settings remained learning rate $3e-4$, 8 environments, 1,024 steps per environment, batch 256, 10 epochs, gamma 0.995, GAE lambda 0.95, and entropy coefficient 0.01.

The run requested 500k and collected **507,904 transitions**, seed 595018.

Metric	Deterministic exam	Stochastic training summary
success	64%	77%
timeout	36%	23%

Metric	Deterministic exam	Stochastic training summary
mean reward	16.80	22.12
mean net progress	18.90	25.26
average episode length	153.80	109.67
forward steps	55.90%	81.98%
stopped steps	44.10%	18.02%
mean throttle	0.417	0.730
collision steps	3.79%	13.80%
mean longest collision streak	5.68	10.30
worst collision streak	41	104

Approximate artifact-time duration: **13.1 minutes**.

Diagnosed exploit

A representative failure kept the target visible for 232 of 318 frames and reacquired it 15 times while making almost no net visible progress. Because PPO_27 awarded **+0.5** on reacquisition, circling across the sight boundary could be profitable. PPO_27 proved that search was learnable, but the reward permitted an orbiting exploit.

What PPO_28 and PPO_29 fixed

These runs sit between the first seeker and the LSTM experiment. Omitting them would hide two important corrections: the orbit reward fix and the deterministic throttle freeze fix.

PPO_28 slow seeker

PPO_28 warm-started PPO_27 and made a coherent control/reward bundle:

- move speed halved from 0.75 to **0.375**;
- turn speed halved from 18 to **9 degrees** to preserve turning radius;
- maximum episode length doubled from 500 to **1,000** to preserve approximate physical search horizon;
- **+0.5** sight reward changed to first-ever sight only;
- exploration rewards switched off permanently after target discovery;
- post-discovery progress became symmetric even while the target was temporarily hidden.

It collected 507,904 transitions and scored **71% deterministic success**, with only 1.90% collision steps. However, **56.73% of deterministic steps were exactly stopped**. Of those stopped steps, 89.76% occurred before first sight, so the dominant problem was pre-discovery freezing rather than obstacle detouring after discovery.

The diagnosed mechanism was an action-distribution mismatch. PPO's Gaussian action mean is unconstrained before clipping, while the throttle space was $[0, 1]$. Negative deterministic means were clipped to exactly zero. Stochastic sampling sometimes crossed above zero, explaining why exploratory training behavior looked more active than deterministic deployment.

PPO_29 normalized throttle: strongest memoryless seeker baseline

PPO_29 changed the policy throttle range to $[-1, 1]$ but mapped it internally to forward-only physical throttle:

```
physical_throttle = (policy_throttle + 1) / 2

-1 -> stopped
0 -> half speed
+1 -> full speed
```

Negative values never caused reverse motion. It also added:

- terminal freeze failure after 60 no-translation steps with one-time -10;
- +0.01 for each newly faced heading bin before first sight, capped at one revolution;
- the existing collision-stuck terminal remained at 40 blocked steps.

PPO_29 warm-transferred PPO_28's actor/value weights into a new compatible-shape MLP with new action bounds and a fresh optimizer. Its 250k run collected **253,952 transitions**.

Metric	PPO_29 deterministic	PPO_29 stochastic
success	69%	70%
timeout	31%	30%
mean reward	16.69	15.91
mean net progress	20.26	22.04
average episode length	170.00	217.88
forward steps	93.71%	89.91%
stopped steps	6.29%	10.09%
mean physical throttle	0.809	0.776
collision steps	4.83%	7.88%
worst collision streak	40	126

Approximate artifact-time duration: **6.6 minutes**. A later 500k confirmation was stopped around 300k because 25-maze checkpoints remained on the same 68-76% plateau.

PPO_29 did not improve success over PPO_28, but it decisively removed the freeze: stopped steps fell from 56.7% to 6.3%. It is the cleanest baseline for asking whether additional memory improves hidden-target discovery.

First LSTM search run: PPO_30

What changed

PPO_30 kept PPO_29's seeker environment, observations, rewards, normalized throttle, freeze rule, scan reward, six-obstacle arena, speeds, and episode limit. The brain changed from Stable-Baselines3 PPO `MlpPolicy` to sb3-contrib `RecurrentPPO("MlpLstmPolicy")`.

- LSTM hidden size: 256
- stacked recurrent layers: 1
- learning rate: $2e-4$
- PPO epochs per rollout: 10
- warm start: none

The recurrent model was trained from scratch because the current training code rejects MLP-to-LSTM warm start.

Exact verified architecture

```
26-number observation sequence
|-- actor LSTM: input 26 -> hidden 256, one layer
|   -> actor MLP: 256->64 -> Tanh -> 64->64 -> Tanh
|   -> action mean 64->2 + learned log_std(2)
|
|-- critic LSTM: input 26 -> hidden 256, one layer
|   -> critic MLP: 256->64 -> Tanh -> 64->64 -> Tanh
|   -> state value 64->1
```

The actor and critic use **separate LSTMs** by default. The LSTM is not a small add-on after the old navigation policy. It replaces the actor and critic input path. The later MLPs now receive 256 recurrent features rather than the original 26 sensors.

Verified parameter count: **623,045**, approximately 52 times PPO_29.

What enters and leaves the LSTM

Every step, both recurrent branches receive all 26 channels:

- target visible;
- target memory age;
- visible/remembered target direction x/z;
- visible/remembered target distance;
- heading sine/cosine;
- previous collision;
- previous throttle and turn;
- all 16 obstacle rays.

The actor LSTM output is transformed into throttle/turn distribution parameters. The critic LSTM output is transformed into a scalar value estimate. Therefore, the recurrent state can potentially encode any combination of target history, obstacle history, prior movement, collisions, headings, and search trajectory.

It does **not** contain named slots such as "target x," "target velocity," or "visited cell." It is a learned distributed state. We cannot truthfully say what each cell stores without additional probing.

How long the memory lasts

There are two different memories:

1. **Explicit target memory:** fixed at 120 environment steps. The environment exposes last-seen direction/distance plus an age channel.
2. **LSTM hidden and cell state:** carried step-to-step until episode end, potentially up to the 1,000-step cap. It resets at the start of each new episode via the episode-start mask.

The LSTM has no simple fixed "120-step window." In principle, its cell state can preserve information for much longer, but useful retention is learned and can decay. Training uses recurrent sequences assembled from PPO rollouts and episode boundaries; it does not guarantee faithful memory for all 1,000 steps.

For one environment, the one-layer recurrent policy carries hidden and cell vectors of length 256 for the actor and another pair for the critic: 1,024 recurrent floating-point state values in total. PPO_31 doubles the layer dimension to 2,048 values.

Results

Metric	PPO_30 200k deterministic	PPO_30 500k deterministic	PPO_29 250k deterministic
collected transitions	204,800	507,904	253,952
success	53%	39%	69%
timeout	47%	61%	31%
mean reward	10.54	3.93	16.69
mean net progress	18.46	10.85	20.26
average episode length	372.80	373.55	170.00
forward steps	99.75%	100%	93.71%
stopped steps	0.25%	0%	6.29%
mean throttle	0.373	0.657	0.809
collision steps	3.58%	4.54%	4.83%
mean longest collision streak	12.05	16.50	8.00
worst collision streak	44	43	40

Stochastic success was 49% at the end of the 200k run and 46% at the end of the 500k run.

Recorded TensorBoard/artifact durations:

- PPO_30 200k: approximately **38.6 minutes**;
- PPO_30 500k: approximately **93.9-95.6 minutes**, depending on whether TensorBoard scalar time or full artifact write time is used.

The 25-maze checkpoint scores oscillated instead of converging reliably. The 200k run was still rising near the end, which justified the 500k test. The independent 500k run later scored only 39% on the rigorous 100-maze exam. This is instability across training time and seed, not evidence that "memory is always harmful," but it is clear evidence that this full recurrent design did not beat the simpler baseline.

Two-layer LSTM run: PPO_31

Changes relative to PPO_30

PPO_31 was not only "PPO_30 with another LSTM layer." It changed:

Setting	PPO_30	PPO_31
LSTM layers	1	2
LSTM hidden size per layer	256	256
exploration reward per new cell	0.01	0.05
reward per new cell/heading view	0.002	0.01
PPO epochs per rollout	10	5
requested transitions	200k/500k	250k

Learning rate remained $2e-4$. Scan reward remained $+0.01$. The rest of the seeker environment matched PPO_30.

Exact verified architecture at the 160k checkpoint

```
26-number observation sequence
|-- actor LSTM: 26 -> 256 -> 256, two stacked recurrent layers
|   -> actor MLP 256->64->64 -> action distribution
|
|-- critic LSTM: 26 -> 256 -> 256, two stacked recurrent layers
|   -> critic MLP 256->64->64 -> value
```

Verified trainable parameters: **1,675,717**. This is about 140 times the memoryless MLP and 2.69 times the one-layer LSTM policy.

What survived from the interrupted run

- requested transitions: 250,000;
- last TensorBoard scalar step: **163,840**;
- last saved checkpoint: **160,000**;
- complete PPO rollouts reached: 20 ($20 * 8,192 = 163,840$);
- elapsed TensorBoard time: **31.9 minutes**;
- checkpoint files: every 10k from 10k through 160k;
- final `leaper_ppo.zip`: missing;
- final 100-episode evaluation: missing;
- TensorBoard non-finite scalars: **0**;
- run JSON status: still `training`, proving normal completion cleanup did not run;
- stderr/exception log: not present.

The correct statement is therefore: **the process ended unexpectedly or was interrupted after completing the 163,840-step rollout, but the stored evidence does not identify the cause.** It would be inaccurate to claim that the neural network numerically exploded.

Deterministic checkpoint curve

Each checkpoint below used 25 deterministic evaluation episodes, so the percentages move in 4-point increments and are noisier than the final 100-episode exams.

Step	Success	Mean reward
10k	0%	-18.46
20k	0%	-26.09
30k	0%	-19.97
40k	0%	-24.55
50k	0%	-21.07
60k	4%	-18.81
70k	8%	-15.22
80k	12%	-7.85
90k	12%	-2.66
100k	24%	4.07
110k	20%	2.67
120k	16%	2.34
130k	24%	3.65
140k	20%	0.03
150k	24%	3.74

Step	Success	Mean reward
160k	20%	1.34

PPO_31 deterministic checkpoint trajectory

Each checkpoint contains 25 episodes; one episode equals four percentage points.

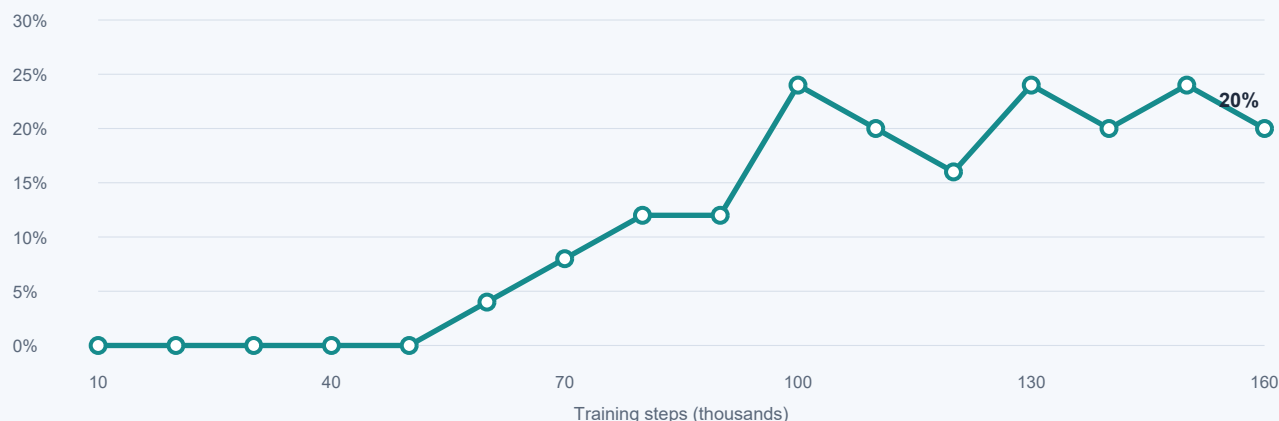


Figure 2. Learning began after 50k, peaked at 24%, and oscillated between 16% and 24% from 100k onward. The run ended after the 163,840-step rollout.

Latest 100 completed stochastic training episodes in the partial diagnostics:

Metric	Value
success	17%
failure/timeout recorded	83%
mean reward	-10.73
mean net progress	8.60
average episode length	643.31
collision steps	9.14%
mean longest collision streak	28.76
worst collision streak	189
forward steps	64.18%
stopped steps	35.82%
mean physical throttle	0.299

Interpretation

The stronger exploration reward did not produce a strong seeker by 160k. It may encourage broad coverage, but it can also make pre-discovery wandering more valuable relative to committing to a search pattern. The partial reward breakdown shows that exploration bonuses were not astronomically large, so reward competition alone should not be assumed to explain all failure.

The two-layer network had substantially more recurrent capacity but also a much harder optimization problem. Because exploration rewards and PPO update epochs changed simultaneously, PPO_31 cannot answer whether two LSTM layers are worse than one.

Why the LSTM appears to "affect the eyes and movement"

This behavior follows directly from the implementation.

Memoryless MLP

```
[target state + heading + prior action/collision + 16 current rays]
|
feed-forward actor
|
throttle and turn
```

Current full recurrent model

```
[target state + heading + prior action/collision + 16 current rays]
|
actor LSTM state
|
actor MLP
|
throttle and turn
```

All obstacle rays pass through the recurrent bottleneck. The action at time t depends not only on the current obstacle geometry but also on the accumulated hidden state. That can be useful for temporal planning, but it also means a poor or unstable memory state can distort an otherwise obvious current obstacle response.

The critic has the same issue separately. Its recurrent state changes the value targets used to train the actor. Thus recurrence can alter both action selection and the learning signal.

The original 64x64 MLP is not sitting intact underneath the LSTM. In the recurrent model, the first actor/critic MLP layers take 256 inputs rather than 26. The old first-layer weights therefore have incompatible shapes. This explains ordinary checkpoint-transfer errors or reports that only some state-dictionary tensors match.

The phrase "two heads" needs precision:

- In actor-critic PPO, the standard heads are the **action head** and **value head**.
- A target-memory system would more accurately be described as a **separate branch/module**, not simply a second output head.

Can navigation and target memory be separated?

Short answer

Yes, with a custom architecture or a hierarchical controller. The current off-the-shelf `MlpLstmPolicy` does not provide that separation automatically.

Why direct checkpoint loading fails

The MLP and recurrent policies do not have the same computation graph:

- MLP actor first layer: 26 inputs to 64 outputs;
- recurrent actor: 26 inputs to an LSTM with 256 hidden units, then 256 inputs to the 64-unit MLP;
- recurrent parameters have no MLP counterpart;
- critic architecture changes in parallel;
- optimizer moments no longer correspond.

This prevents a simple "load PPO_25 and add LSTM" operation. It does not prevent deliberate module-level reuse.

What can realistically be preserved

1. **Preserve behavior by architecture.** Keep a local-navigation module with the same obstacle-ray and desired-goal interface it already understands.
2. **Preserve weights selectively.** Reuse compatible later layers or the entire local controller while initializing only the tracker/planner branch.

3. **Freeze in stages.** Initially freeze the navigation actor while training the tracker to provide a compatible desired direction. Later unfreeze lightly for joint fine-tuning.
4. **Use distillation.** During training, penalize deviation from the old policy on fully informed navigation states while also learning the new search/tracking task.
5. **Retrain the critic.** Even when the actor's local-navigation behavior is preserved, the value function should adapt to the new partially observable and moving-target task.

Important limitation

PPO_25's actor was trained for a fixed, always-known target. It may be an excellent local goal follower, but it has no learned concept of "target not found," belief uncertainty, or search mode. Preserving all of it unchanged would also preserve those limitations. The better goal is to preserve the obstacle-avoidance/local-steering competence, not freeze every weight indiscriminately.

State augmentation or LSTM?

What state augmentation means here

State augmentation explicitly calculates useful temporal features outside the policy and appends them to the observation. The seeker already does this with:

- target-visible flag;
- time since target was seen;
- direction and distance to last-seen target position.

For a moving target, this can be extended with an explicit belief state:

- estimated target world position;
- estimated target velocity x/z ;
- age since last reliable sighting;
- confidence/uncertainty;
- whether the estimate is observed or predicted;
- optional intercept direction rather than only current predicted direction.

An alpha-beta filter or Kalman-style tracker could update these values when the target is visible and propagate them while it is hidden. This is interpretable, testable, and cheap.

What state augmentation can solve well

- short and medium occlusions;
- smooth target motion;
- explicit last-seen position and velocity;
- confidence decay;
- deterministic debugging of belief error;
- retaining the current MLP architecture with limited input changes.

What it does not solve automatically

- deciding which unexplored region to search next;
- remembering a large detailed map unless a map is explicitly supplied;
- handling adversarial or highly unpredictable target motion;
- learning complex behavior from raw visual sequences if the tracker itself cannot identify the target.

Does using state augmentation now block LSTM later?

No. State augmentation and LSTM are complementary. A future recurrent planner can consume the same explicit belief features. Starting with an interpretable target tracker does not lock the project out of recurrence.

The dangerous transition is not "state augmentation now, LSTM later." The dangerous transition is replacing a stable policy with an entirely new recurrent actor-critic and expecting its old behavior to survive automatically. A modular interface avoids that.

Practical recommendation

For the next moving-target stage, explicit target belief augmentation should be the first baseline. It directly represents the information the game needs and allows exact measurement of tracking error. A small isolated GRU/LSTM can then be compared against it on the same task.

Possible setup going forward

Option A: explicit target tracker plus MLP controller

```
semantic sight measurement
  (visible, bearing, range)
    |
  explicit motion tracker
  (position, velocity, age,
   confidence, prediction)
    |
  obstacle rays + robot state + target belief
    |
  feed-forward PPO controller
    |
  throttle and turn
```

Advantages:

- smallest change;
- interpretable memory;
- easy unit tests;
- current seeker already contains part of it;
- suitable for a moving target with reasonably smooth motion.

Risk: the MLP still has to combine search and pursuit in one controller, and a compact belief may not encode search coverage.

Option B: target-only recurrent branch

```
target measurement stream + robot motion delta
    |
  small GRU/LSTM tracker
    |
  target belief feature vector
    |
current obstacle rays ---> navigation fusion/controller ---> actions
```

Recommended tracker inputs:

- visible flag;
- observed relative target x/z or bearing/range when visible;
- robot translation and yaw change since prior step;
- elapsed time;
- age/confidence mask.

Do **not** feed all 16 obstacle rays into this target tracker unless an ablation proves they help infer occlusion geometry. Current obstacle sensing can remain on a feed-forward path.

A one-layer 32- or 64-unit GRU/LSTM is a more proportionate first test than two 256-unit layers. The tracker output can be concatenated with navigation features. The controller can be initialized from PPO_29 where shapes permit or trained with a preservation/distillation objective.

Option C: hierarchical search planner plus frozen local navigator

```
target tracker + search coverage state
|
high-level planner (slow cadence)
outputs waypoint / desired bearing
|
pretrained local obstacle navigator (fast cadence)
|
throttle and turn
```

This most closely matches the owner's desired separation:

- local navigator answers "how do I safely move toward this waypoint?";
- planner answers "where should I search or intercept next?";
- tracker answers "where is the target probably located now?"

For this to work cleanly, the local navigator should be trained or validated on arbitrary waypoint directions, not only PPO_25's one fixed world target. PPO_25 is an excellent donor and teacher but should not be assumed to be a universal waypoint controller without testing.

Recommended choice

Use Option A as the interpretable baseline, then test Option B or C. Do not make full two-layer recurrent PPO the default merely because the eventual target moves.

Why the LSTM runs may have performed badly

These are evidence-based hypotheses, not proven single causes.

1. **The recurrent model was 52-140 times larger.** The same amount of on-policy data now had to fit far more parameters.
2. **It relearned the entire controller from scratch.** The strongest navigation policy was not retained as an intact module.
3. **The LSTM processed instantaneous obstacle rays unnecessarily.** Recurrence could contaminate a sensor-to-action response that worked well feed-forward.
4. **The task mixes modes.** Search, first detection, remembered pursuit, collision escape, and timeout avoidance all share one recurrent state and one action head.
5. **PPO recurrent optimization was unstable.** PPO_19-21 and PPO_30 showed oscillating checkpoint quality. A later checkpoint was not necessarily better.
6. **A 25-episode checkpoint is noisy.** Four percentage points equal one episode, so apparent spikes are not robust.
7. **The environment already supplied 120-step target memory.** The LSTM's incremental benefit had to come mainly from remembering search coverage or longer context, a harder and less directly rewarded job.
8. **Search coverage is hidden from the observation.** The reward system knows visited cells and views, but the policy is not directly given a visited map. The LSTM must infer coverage indirectly from long sequences of headings/actions/rays.
9. **PPO_31 mixed multiple interventions.** Deeper memory, stronger exploration reward, and fewer PPO epochs prevent causal diagnosis.

10. **The exploration reward may compete with commitment.** Before first sight, larger novelty bonuses can encourage continual coverage instead of stable systematic search. After sight they switch off, which is correct, but pre-sight behavior can still be distorted.

Simple next experiments

Phase 0: close the PPO_31 evidence gap

Do not blindly resume training. First:

1. run the saved 160k checkpoint through the full fixed 100-episode deterministic evaluation;
2. preserve the checkpoint and TensorBoard data;
3. reproduce the process under captured stdout/stderr if diagnosing the interruption matters;
4. record system memory/GPU/CPU usage and process exit code;
5. do not call the interruption a neural crash unless an exception, out-of-memory event, or non-finite value is observed.

Given its 20% 25-maze checkpoint score and 17% latest stochastic success, continuation is unlikely to become the preferred architecture without a strong new reason.

Phase 1: establish the correct baseline

Use PPO_29 as the hidden-target policy baseline, not PPO_25's 93% score. Retain:

- 16 rays, 270 degrees, range 28;
- normalized forward-only throttle;
- 60-step freeze terminal;
- 40-step collision-stuck terminal;
- first-sight-only reward;
- exploration rewards disabled after discovery.

Run at least three seeds if a change appears promising.

Phase 2: explicit moving-target belief baseline

Introduce a target that moves slowly and predictably. Add explicit state:

- current observed target-relative position when visible;
- estimated world velocity;
- predicted relative position while hidden;
- age and confidence.

Measure belief error separately from control success. This tells whether failure comes from tracking or navigation.

Phase 3: modular recurrent ablation

Compare, one change at a time:

Variant	Memory	Navigation path
A	explicit position/velocity tracker only	existing MLP
B	small target-only GRU/LSTM	feed-forward obstacle/navigation branch
C	high-level recurrent planner	pretrained local waypoint navigator
D	full MlpLstmPolicy	all inputs recurrent, current PPO_30 style

Keep reward, arena, target motion, seed set, and training budget identical across variants.

Phase 4: curriculum for moving pursuit

1. target visible and stationary;
2. target visible and moving slowly;
3. moving target with short occlusions;
4. longer occlusions;
5. denser obstacles;
6. more varied target motion;
7. full game distribution.

Only advance when deterministic evaluation, not stochastic training success, is stable.

Phase 5: metrics beyond success

Add these to the evaluation report:

- probability of first detection;
- time to first detection;
- success conditional on detection;
- reacquisition probability after occlusion;
- time to reacquisition;
- target belief position error while visible and hidden;
- target velocity estimation error;
- fraction of arena cells/views covered before detection;
- redundant revisit rate;
- interception time after first sight;
- collision and freeze rates;
- deterministic versus stochastic action gap;
- success across target speed and occlusion duration buckets.

These metrics will reveal whether a failure is search, memory, prediction, or local navigation.

Questions for Fable

The following are the owner's core questions, rephrased precisely.

1. **Can this project preserve its learned local-navigation policy while adding a separate memory mechanism for target search and tracking?** Please distinguish what is impossible with the current off-the-shelf `MlpLstmPolicy` from what is possible with a custom modular or hierarchical policy.
2. **What architecture would best isolate responsibilities?** Should obstacle rays and immediate locomotion remain feed-forward while only target observations, target motion, and search history enter a recurrent branch?
3. **Is a "two-head" design the right concept, or should this be expressed as a tracker, planner, and controller hierarchy?** Please specify where actor, critic, recurrent state, and navigation features should live.
4. **Which pretrained weights can safely be reused?** PPO_25 has excellent known-target navigation; PPO_29 is the strongest hidden-target MLP. Which is the better donor, which layers should be frozen, and which must be retrained?
5. **How should the observation interface be designed so transferred navigation weights retain meaning?** In particular, how should target-visible, target age, predicted direction/distance, velocity, heading, previous action, collision, and 16 rays be routed?

6. **Should the next baseline use explicit state augmentation rather than LSTM?** Would last-seen position, estimated velocity, confidence, and prediction through occlusion be sufficient for the first moving-target stages?
7. **Does choosing state augmentation now create a future migration problem?** If an LSTM becomes necessary later, can it consume the augmented belief state without destroying the existing navigation behavior?
8. **What should recurrent memory actually receive?** Should it receive only target measurements and robot ego-motion, or also obstacle rays and collision/action history? What information is necessary for target motion inference versus map/search memory?
9. **What recurrent size and type are proportionate?** Would a one-layer 32/64-unit GRU or LSTM be more appropriate than the current separate 256-unit actor and critic LSTMs? Should actor and critic share a tracker state?
10. **How should memory be trained and reset?** What sequence length, burn-in, truncation strategy, episode-start handling, and curriculum are appropriate for occlusions lasting tens to hundreds of steps?
11. **Why did full recurrence underperform here?** Given the verified architecture and results, which failure mode is most likely: optimization instability, excessive capacity, full-policy relearning, reward competition, insufficient sequence training, lack of explicit search state, or another issue?
12. **How should PPO_31 be interpreted?** It changed LSTM depth, exploration rewards, and PPO epochs simultaneously and stopped at 163,840 without a recorded exception. What clean ablations should replace it?
13. **How can navigation behavior be protected during new-task learning?** Please evaluate staged freezing, behavior cloning/distillation from PPO_25/PPO_29, auxiliary action-preservation loss, separate learning rates, or hierarchical control.
14. **Should the critic be recurrent even if the actor's local-navigation path is feed-forward?** Conversely, can a target tracker be shared while actor and critic heads remain task-specific?
15. **What is the best search-memory representation?** Is an egocentric visited-cell map, frontier direction, compact coverage vector, recurrent hidden state, or external map most suitable for the current 62.5-unit arena?
16. **How should the moving target be represented?** Should the controller pursue predicted current position, estimated intercept point, or a high-level waypoint? How should uncertainty change behavior when the target remains hidden?
17. **How should rewards change for moving pursuit without creating exploits?** Please address first detection, reacquisition, progress caused by Leaper versus progress caused by target motion, exploration bonuses, time cost, belief accuracy, and interception reward.
18. **What evaluation protocol would demonstrate that memory genuinely helps?** Please propose fixed seeds, multiple training seeds, target-motion distributions, occlusion-duration buckets, and statistical comparisons against PPO_29 and explicit-state baselines.
19. **Should the project abandon full-policy LSTM for the static-target stage but retain recurrence for moving targets?** If so, what evidence threshold should trigger adding recurrence?
20. **What is the smallest next experiment that most reduces uncertainty?** Please give one concrete architecture, one controlled comparison, recommended budget, and a go/no-go criterion.

Facts that should constrain Fable's answer

- The 93% PPO_25 score is a known-target navigation score, not a search score.
- The current seeker already has explicit 120-step last-seen target memory.
- The strongest memoryless hidden-target score is PPO_29 at 69% deterministic.
- PPO_30's one-layer LSTM completed; it did not numerically crash. It scored 53% at 200k and 39% at 500k.
- PPO_31 stopped at 163,840, but no exception log survives and TensorBoard has no non-finite scalar.
- PPO_31 was a compound experiment, not an isolated test of two LSTM layers.
- Current recurrent PPO sends all 26 channels through separate actor and critic LSTMs.

- The recurrent policy is dramatically larger than the MLP: 623k or 1.676M parameters versus 11.973k.
- The obstacle-navigation competence is real and well replicated, but exact policy transfer across a changed observation meaning and architecture is not automatic.
- Deterministic 100-episode evaluation is the score of record. Small 25-episode checkpoint peaks and stochastic training success must remain separate.

Files used

Primary run artifacts:

- `rl_artifacts/ppo_25_idle_s3/training_config.json`
- `rl_artifacts/ppo_25_idle_s3/final_evaluation.json`
- `rl_artifacts/ppo_25_idle_s3/leaper_ppo.zip`
- `rl_artifacts/ppo_27_seeker_500k/training_config.json`
- `rl_artifacts/ppo_27_seeker_500k/final_evaluation.json`
- `rl_artifacts/ppo_28_slow_seeker_500k/training_config.json`
- `rl_artifacts/ppo_28_slow_seeker_500k/final_evaluation.json`
- `rl_artifacts/ppo_29_normalized_throttle_250k/training_config.json`
- `rl_artifacts/ppo_29_normalized_throttle_250k/final_evaluation.json`
- `rl_artifacts/ppo_30_lstm_search_200k/training_config.json`
- `rl_artifacts/ppo_30_lstm_search_200k/final_evaluation.json`
- `rl_artifacts/ppo_30_lstm_search_500k/training_config.json`
- `rl_artifacts/ppo_30_lstm_search_500k/final_evaluation.json`
- `rl_artifacts/ppo_31_lstm_explore_250k/training_config.json`
- `rl_artifacts/ppo_31_lstm_explore_250k/training_diagnostics.csv`
- `rl_artifacts/ppo_31_lstm_explore_250k/browser_replay.json`
- `rl_artifacts/ppo_31_lstm_explore_250k/checkpoints/leaper_ppo_160000.zip`
- `rl_artifacts/ppo_31_lstm_explore_250k/tensorboard/`

Primary implementation and project records:

- `rl_environment.py`
- `train_rl.py`
- `TRAINING.md`
- `RL_SPEC.md`
- `RUN_LOG_PP023-26.md`
- `src/brain/leaperWorld.js`

Repository inventory at the time of drafting: **62 run directories**, of which **44 contain final evaluation files**. The remainder include smoke tests, partial runs, and interrupted/aborted experiments. Run-directory count is therefore not the same as the number of completed major experiments.

All reported metrics were reconstructed from saved configurations, evaluations, checkpoints, diagnostics, TensorBoard events, and the implementation source.